Relazione JBomberman

Gabriele Pelissetto 2045067 canale MZ

Caratteristiche del gioco:
numero di nemici: 8 i cui comportamenti diversi sono codificati in *src/main/resources/app/jbomberman/json/entitiesSpecs.json*
numero di powerups: 8 che conferiscono a Bomberman diverse capacita' codificate in *src/main/resources/app/jbomberman/json/entitiesSpecs.json* (ad esempio conferiscono un maggior numero di bombe, la possibilita' di attraversare i muri, l'invincibilita', la possibilita' di controllare a distanza l'esplosione delle bombe, la resistenza al fuoco, una maggior velocita' di movimento, un maggior range delle bombe, la possibilita' di passare sopra le bombe)
numero di livelli: 50 codificati in *src/main/resources/app/jbomberman/json/levels.json*
libreria grafiche: sia JavaFX che java swing (e' necessario editare un file di configurazione per scegliere la libreria da utilizzare)
dipendenze da librerie esterne: *org.json:json* per la gestione di *JSON*, *org.apache.logging.log4j* per il logging. Si e' utilizzato *gradle* per la gestione delle dipendenze e per il processo di compilazione del programma.
Piattaforme supportate: *Windows*, *Linux*. Il programma e' anche stato pensato per girare su *Mac OS X*, ma non l'ho testato perche' non dispongo di un computer con tale sistema operativo. Funziona anche su *FreeBSD*, *OpenBSD*, *NetBSD* anche se e' stato necessario ristrutturare leggermente il codice. *JavaFX* non e' supportato su piattaforme *BSD* e quindi il codice *javafx*, deve ivi essere rimosso. L'implementazione mediante java swing, invece, funziona sui sistemi *BSD*.

Per lanciare l'applicazione e' necessario eseguire *make run.*

Gestione del profilo utente
La gestione del profilo utente avviene in modo molto semplice. La prima scena del gioco presenta un menu', tra le cui entrate ve ne e' una con la scritta **LOGIN**. Selezionandola, si apre una schermata che permette la creazione di nuovi utenti, di effettuare il login in un account e di settare un avatar. Una volta creato un accout, esso viene registrato in un file json, posto in *%APPDATA%\jbomberman\players.json* su Windows, *~/.config/jbomberman/players.json* su sistemi Linux, BSD e *~/Library/Application Support/jbomberman.json* su OS X. Al momento del login, viene letto il file di configurazione e creata una instanza della classe *app.jbomberman.model.game.entities.BomberManCapabilities* che mantiene il conteggio delle vite, il punteggio del giocatore e i powerup raccolti. Durante ogni partita, viene aggiornato l'oggetto istanza della classe *BomberManCapabilities* e, quando avviene un altro login oppure viene chiusa l'applicazione, viene anche aggiornato il file di configurazione.
Per quanto riguarda l'avatar, per il caricamento delle immagini, viene aperta una finestra che permette all'utente di selezionare l'avatar desiderato tra le immagini disponibili nel filesystem, controllando che il file selezionato sia effettivamente una immagine. L'avatar viene poi, qualora presente, mostrato accanto al nome dell'utente durante la partite.

Descrizione di una partita completa
Una volta lanciata l'applicazione, si apre una schermata iniziale (sita in *app.jbomberman.view.scenes.StartDisplay.java*) la quale presenta una menu' con tre entrate: **START**, **LOGIN**, **EXIT**. **LOGIN** serve per entrare nella schermata di login e **START** avvia invece la partita. Non e' comunque necessario effettuare il login per avviare una partita. Qualora sia stato effettuato un login, viene cariacato il livello che quel giocatore ha raggiunto, altrimenti viene caricato il primo livello.
Avviene dunque una transizione alla schermata modellata da *app.jbomberman.view.scene.LoadingLevelScene* che mostra il numero del livello, riproduce il suono

iniziale e, infine, vi e' una transizione alla schermata GameScene (*app.jbomberman.view.scene.GameScene*) che da' avvio al gioco. Viene, a questo punto, sollecitato il *Model* a creare una nuova partita modellata dalla classe *app.jbomberman.model.game.Game*. La classe *Game* e' responsabile della creazione di una nuova *GameGrid,* che e' una griglia la quale contiene le Entita' che costituiscono il gioco (nemici, powerups, bomberman, blocchi, porte, muri, ...). Per la sua costruzione, e' usato il *builder pattern*, come specificato in Design Patterns[1]. E' stata creata una classe *GameGridBuilder* e una classe *GameGridBuilderDirector* che devono cooperare per la creazione della *GameGrid*. *GameGridBuilderDirector* legge il file di configurazione *src/main/resources/app/jbomberman/json/levels.json* per determinare quali componenti deve avere il livello che si sta costruendo (quanti muri ci devono essere, quanti nemici, quali nemici, quali powerups...). *GameGridBuilder*, invece, deve inserire nella griglia tutte le diverse entita'. In questo modo abbiamo separato il codice che aggiunge le entita' alla *GameGrid* in *GameGridBuider* da quello che invece gestice la lettura e la decodifica del file *JSON src/main/resources/app/jbomberman/json/levels.json* in *GameGridBuilderDirector*. In piu', *GameGridBuilder*, per essere il piu' *JSON-indipendente* possibile, crea le entita' mediante la classe *EntityFactory* (che permette di creare nemici, powerups, ...). Essa usa il *singleton pattern*, poiche' vi deve essere un'unica istanza della classe *EntityFactory* durante l'esecuzione del programma. Essa costruisce le diverse entita' leggendone le specifiche da un file *json*, che si trova in *src/main/resources/app/jbomberman/json/entitiesSpecs.json*. Dopo la costruzione della *GameGrid,* il gioco comincia. E' possibile sospendere il gioco premendo ESC; appare un menu' (codificato da *app.jbomberman.view.scene.PauseScene*) che permette di uscire definitivamente o di riprendere il gioco. Per l'implementazione dei diversi comportamenti delle entita', e' stato impiegato lo *state pattern* come descritto in Design Patterns[1]. Ad esempio, il comportamento di un nemico (*app.jbomberman.model.game.entity.Enemy*) cambia a seconda dello stato in cui egli si trova: se e' vivo, il suo stato e' *app.jbomberman.model.game.state.EnemyAliveState,* se e' stato colpito dall'esplosione di una bomba, si trova nello stato *app.jbomberman.model.game.state.EnemyDeadState*. Mediante la delegazione, sono gli stati che quindi determinano il comportamento del nemico. Lo stesso vale anche per i muri. Quindi in *app.jbomberman.model.game.state* vi e' l'insieme di Stati in cui le diverse entita' si possono trovare. Il gioco termina quando Bomberman muore (ossia viene colpito da una sua bomba oppure viene catturato da un nemico) oppure quando Bomberman uccide tutti i nemici, trova la porta d'accesso al prossimo livello e la apre andandoci sopra. Nel caso di una sconfitta, viene ricaricato lo stesso livello se il numero di vite ancora disponibili lo permette, oppure viene mostrata la schermata di game over e il gioco ritorna alla schermata iniziale. In caso di vittoria, il gioco prosegue al livello successivo o, se il livello successivo e' il 51-esimo, mostra la schermata di fine gioco e ritorna alla schermata iniziale. I 50 livelli sono stati generati automaticamente mediante lo script sito in *utils/mk_level.rb* che genera il file *src/main/resources/app/jbomberman/json/levels.json* il quale contiene le specifiche dei livelli.

Descrizione dei design patterns utilizzati

Abbiamo utilizzato il design pattern MVC, crendo tre differenti pacchetti (model, view, controller). Il *model* contiene la logica del gioco, la *view* la gestione dell'interfaccia grafica e le scene, e il *controller* e' responsabile del setup della *View*.

Il pacchetto *controller/* contiene un'unica classe *Controller* la quale ha lo scopo di inizializzare la *View* e in piu', ad esempio, puo' essere interpellato dal *Model* per aggiungere un observer a *KeyBoardController,* istanza di *Observable* che e' responsabile della gestione degli eventi generati dalla tastiera. In questo modo, la creazione del rapporto *Observer/Observable* tra *Model* e *View*, avviene mediante il *Controller*.

Il pacchetto *model/* contiene invece la logica del gioco. Internamente (nella classe *app.jbomberman.model.game.Game*) viene utilizzato un clock a 60Hz (modellato dalla classe *app.jbomberman.util.Timer)* per determinare quanto spesso il gioco deve essere aggiornato (~60 volte al secondo).

Nel pacchetto *view/* vi e' il codice per la gestione dell'interfaccia grafica. Nel corso della progettazione di *JBomberman*, si e' deciso di valutare l'astrattezza e flessibilita' del codice fornendo una doppia implementazione della interfaccia grafica. Una utilizza *java swing* e l'altra usa *javafx*. Per passare dall'una all'altra e' solo necessario modificare il file di configurazione *src/main/resources/app/jbomberman/json/settings.json*, associando al campo "GRAPHIC_LIBRARY": la stringa "swing" o "javafx". Un ulteriore motivo per questa scelta, e' stato il fatto che l'implementazione iniziale utilizzava solo javafx, il quale si e' scoperto non essere supportato su alcuni sistemi (come BSD). Quindi, nella *View*, per rendere piu' semplice l'implementazione, abbiamo utilizzato il *Factory Pattern*. Infatti abbiamo astratto i componenti specifici delle librerie grafiche (come ad esempio AnimationTimer o *javax.swing.Timer*), li abbiamo incapsulati in classi wrapper (ad esempio *app.jbomberman.view.graphics.javafx.JavaFXLoopTimer* e *app.jbomberman.view.graphics.swing.SwingLoopTimer*, sottoclassi della classe astratta *app.jbomberman.view.LoopTimer*), cosi' da permettere al *Controller* e alle scene di gestire classi astratte. Essi puossono ottenere delle loro istanze mediante il riferimento ad una concretizzazione della classe astratta *app.jbomberman.view.GraphicFactory*, contenuto nella *View*. Le concretizzazioni sono *app.jbomberman.view.graphics.swing.SwingGraphicFactory* e *app.jbomberman.view.graphics.javafx.JavaFXGraphicFactory* e forniscono metodi per la creazione di classi *app.jbomberman.view.LoopTimer* e *app.jbomberman.view.KeyBoardController*.
Similmente accade nel pacchetto *image/*. Anche in questo caso, abbiamo utilizzato il *Factory pattern*. Vi e' una classe *FlightWeightImagesFactory* che e' la *factory* delle immagini. La *factory* costruisce delle *AbstractImage* e utilizza per far cio' un'istanza concreta di *AbstractImageBuilder*. Per l'utilizzo di *java swing* la concretizzazione di *AbstractImageBuilder* e' *app.jbomberman.image.swing.SwingImageBuilder* che produce delle *app.jbomberman.image.swing.SwingImage* incapsulanti delle *BufferedImage*. Similmente avviene per *javafx* (*app.jbomberman.image.javafx.JavafxImageBuilder* che produce *app.jbomberman.image.javafx.JavaFXImage* incapsulante *javafx.image.Image*). In piu', nel pacchetto *image*, usiamo anche il *flightweight pattern* e il *singleton pattern*. E' inutile caricare una immagine piu' volte, essendo piu' conveniente caricare le immagini una sola volta e porle in una mappa che associa ad un path (locazione su disco) una immagine, cosi' da salvare spazio in memoria e evitare inutili accessi al disco. E' possibile far cio' perche' le immagini sono immutabili e possono quindi essere condivise da diverse classi. Usare il *singleton pattern* e' necessario proprio perche' vi e' l'utilizzo del *flightweight pattern*. Per effetture il caching delle immagini, vi deve essere una sola istanza di *FlightWeightImageFactory*.

In piu' vi sono anche altri pacchetti quali *util/*, *audio/*, *image/* e *drawing/*.
Il pacchetto *util/* contiene delle classi di utilita' che vengono utilizzate da tutti gli altri pacchetti. In esso, vi e' una diversa implementazione del pattern *Observer/Observable*. *Java* fornisce una classe deprecata *Observable* e un'interfaccia deprecata *Observer*. Oltre ad essere deprecata, la classe *Observable* ha un altro difetto: e' una classe e non un'interfaccia. Infatti, dato che *Java* non possiede l'ereditarieta' multipla per evitare il problema del diamante, e' impossibile creare una classe osservabile e a sua volta sottoclasse di una classe arbitraria. Bisogna tenere conto che questa scelta ha anche dei vantaggi, come quello di non dover definire in ogni sottoclasse di *Observable* una collezione contente i riferimenti agli osservatori.
Nella mia implementazione, anche *Observable* e' un'interfaccia. Ovviamente questa implementazione ha il problema che ogni osservabile deve definire una collezione per mantenere i riferimenti agli osservatori e un metodo *getObservers()* per ritornare quella collezione. Per ovviare a questo problema, ho creato la classe astratta *AbstractObservable*, che implementa l'interfaccia *Observable* e fornisce gia' il metodo *getObservers()* implementato. Quindi, qualora si debba implementare un osservabile la cui unica superclasse e' *Object*, si puo' estendere *AbstractObservable*; se invece si vuole implementare un osservabile che abbia altre superclassi, si deve implementare l'intefaccia *app.jbomberman.util.Observable* e definire il metodo *getObservers()*.

Un’altra classe fondamentale e’ *Maybe<T>*. Essa prende ispirazione dal tipo *Maybe* del linguaggio di programmazione funzionale *Haskell*. L’idea di base e’ che, in *Java,* quando una variabile non contiene alcun riferimento ad un oggetto, contiene il valore *null*. Quindi, dato che tutte le variabili potrebbero contenere il valore *null*, bisognerebbe, prima di accedere a qualsiasi loro metodo, controllare che la variabile non sia *null*. Lavoro tedioso e per la maggior parte inutile, perche’ la stragrande maggioranza delle variabili e’ inizializzata e non e’ mai *null*. Quindi il controllo (variabile != *null*) deve essere inserito dal programmatore solo quando c’e’ la possibilita’ che un oggetto contenga *null*. Il programmatore si deve ricordare quindi quali oggetti possono essere *null* e questo chiaramente introdurra', senza dubbio, prima o poi degli errori. Quindi ho creato le variabili che potrebbero contenere *null* come istanze della classe *Maybe*. Un'istanza della classe *Maybe* puo’ trovarsi in due stati, *nothing* e *just*. Quanto si trova nello stato *nothing*, vuol dire che non c’e’ nessun riferimento ad un oggetto (e' come se contenesse *null*); quando e’ *just*, mediante l’uso del metodo *fromJust(),* si puo’ ottenere il valore contenuto. Questo obbliga il programmatore, ogni volta che deve prendere un valore da una variabile *Maybe*, a controllare che la variabile contenga qualche valore possibilmente evitando errori che potrebbero insorgere dall’uso di semplici variabili contenti *null*. Quindi, in ogni istante durante l'esecuzione del programma, tutte le variabili sono sempre inizializzate e diverse da *null*. In piu’, la classe *Maybe* contiene dei metodi che permettono di impiegare un paradigma di programmazione funzionale in *java*. Ad esempio, *bindM()* emula il comportamento di (>>=) in *haskell* ossia, qualora l’oggetto su cui venga chiamato sia *nothing*, ritorna *nothing*, altrimenti chiama su di esso *fromJust*() e lo passa in input alla funzione passata come argomento a *bindM* e ritorna cio’ che essa ritorna. Avremmo potuto usare la meno potente classe *Optional* fornita dalla libreria standard che pero’, essendo *final*, non puo’ essere estesa a piacimento.
Un altra classe fondamentale e’ la classe *Vector2D*. Nella logica del gioco, l’utilizzo di coppie di coordinate avviene in maniera massiccia e quindi e’ stato conveniente durante la progettazione del gioco utilizzare *Vector2D* per mantenere accopiate una coppia di coordinate. L’unico problema e’ l’overhead di tutto questo. Creare un oggetto *Vector2D* ogni volta che si ha a che fare con delle coordinate vuol dire che la *JVM* sara' costretta ad allocare molti oggetti sulla *heap* e molto lavoro dovra' anche compiere il *garbage collector*. Per ridurre questi overhead (in particolare quello legato alla creazione di *Vector2D*), e’ stato usato il *Flightweight pattern* similmente a come descritto in *Design Patterns*[1], ma in modo leggermente piu’ semplice. *Vector2D* e’ il *Flightweight*, ma non abbiamo una vera e propria *FlightweightFactory*, perche’, per ottenere una istanza di un *Vector2D*, e’ necessario chiamare il metodo statico *Vector2D.of(int x, int y).* Essendo le instanze di *Vector2D* immutabili (non ci sono *setters* pubblici), i *Vector2D* possono essere condivisi da piu’ oggetti all’interno della applicazione. Quindi il metodo *of()* guarda in una mappa per vedere se eventualmente un *Vector2D* con le stesse componenti sia gia’ stato creato. In tal caso lo ritorna, altrimenti crea un nuovo *Vector2D* e lo aggiunge alla mappa. Ho effettuato dei testi e il rateo di hit in questa mappa e' molto buono. Attorno al 99.99780190196664% durante il gioco della prima partita (*#Hit*: 44128090, *#Miss*: 970).
Abbiamo anche creato *MutableVector2D* che permette di creare istanze mutabili di *Vector2D* (per le quali pero' non viene usato il flightweight pattern)
Il pacchetto *drawing/* definisce tre importanti classi quali *DrawObject*, *LayeredDrawObject* e *Drawing*. *DrawObject* rappresenta una immagine da disegnare. Esso incapsula una *AbstractImage* e una coordinata che ne specifica la posizione. *LayeredDrawObject* e' un *DrawObject* che pero' contiene anche l'indicazione del *layer* sul quale deve essere disegnato, rappresentato come un intero. Minore e' l'intero, prima verra' disegnata l'immagine. Cio' serve nel caso in cui si debba lavorare con immagini che si sovrappongono, in modo da definire un ordine nel quale le immagini debbano essere disegnate. *Drawing* invece contiene dei *DrawObjects* e, una volta creata una sua istanza dal Model, viene passata alla View. Essa ha il compito di rappresentare il disegno mediante la libreria grafica.

Descrizione delle classi per la riproduzione di audio sample

Per la riproduzione degli *audio sample*, e' stato progettato il pacchetto *audio/*, che contiene una mia implementazione di un flessibile audio manager. *AudioPlayer* e' la classe principale che deve essere invocata per la gestione dell'audio. Utilizza il *singleton pattern* poiche' vi deve essere un'unica istanza di *AudioPlayer* durante l'esecuzione dell'applicazione. Ogni volta che si vuole lanciare la riproduzione di un suono, si usa il metodo *play()* specificando il file da riprodurre. Vi sarebbe un notevole overhead, qualora non venisse fatto il caching dei file audio caricati e dovessimo continuamente aprire e caricare i file. Quindi manteniamo una *soundPool* che e' una mappa che ha come chiavi i nomi dei file, e come valori i loro contenuti (di tipo *byte[]*). In questo modo rendiamo molto piu' veloce la riproduzione perche' il tempo necessario per il caricamento viene quasi del tutto annullato. Ad esempio, quando durante una partita vengono posizionate molte bombe ~100, e poi il giocatore decide di farle esplodere tutte nello stesso momento tenedo premuto il carattere b, senza questo accorgimento la *JVM* avrebbe dovuto aprire 100 volte lo stesso file, leggerlo 100 volte causando un *overhead* non indifferente.
La riproduzione dei suoni e' affidata alla classe *ThreadPlayer*. Essa estende l'interfaccia *Runnable* e quindi puo' essere eseguita da un *Thread* differente rispetto al *Main Thread*. Cio' e' necessario per permettere al gioco di proseguire anche mentre un suono viene riprodotto.
Dal punto di vista delle performance va anche notato che per lanciare i *ThreadPlayer* che devono riprodurre il suono, viene usato un oggetto istanza della classe *java.util.concurrent.ExecutorService*, chiamato *executorsPool* (in *app.jbomberman.audio.AudioPlayer*), che viene inizializzato mediante *Executors.newCachedThreadPool()*. Facendo cio', l'overhead dovuto alla creazione di nuovi threads che riproducono i suoni viene quasi del tutto eliminato, in quanto ogni *thread* che esegue un *ThreadPlayer*, una volta terminata l'esecuzione, non viene distrutto, ma viene mantenuto pronto per eseguire nuovi *ThreadPlayers*.
Il metodo *play()* di *AudioPlayer* ritorna un *AudioHandle*, che e' un oggetto il cui unico compito e' quello di fare da tramite tra i *client* che chiedono di riprodurre i suoni e il *ThreadPlayer* che effettivamente riproduce il suono.

Adozione degli *stream*
Gli *Stream* sono stati utilizzati ove possibile (in
./src/main/java/app/jbomberman/drawing/Drawing.java
./src/main/java/app/jbomberman/model/game/entity/FreeCell.java
./src/main/java/app/jbomberman/model/game/GameGridBuilderDirector.java
./src/main/java/app/jbomberman/model/game/MoveManager.java
./src/main/java/app/jbomberman/view/View.java). Ad esempio in
src/main/java/app/jbomberman/view/View.java, utilizziamo gli *stream* per calcolare la dimensione di un disegno, ossia il vettore che ha come coordinate i massimi delle coordinate dei *Vector2D* dei *Drawing*.

Animazioni ed effetti speciali
Le animazioni sono implementate mediante l'utilizzo dello *state pattern*. Questo permette di cambiare l'aspetto delle entita' del gioco a seconda del loro stato. In tal modo, per esempio, nel momento in cui Bomberman o dei nemici vengono uccisi, vengono mostrate delle ulteriori immagini che rendono piu' evidente la morte del nemico o di Bomberman stesso. Sono state anche aggiunte delle strisce rosse che simulano il fuoco dell'esplosione delle bombe e anche le dissolvenze dei muri colpiti.

Note per l'UML:
Abbiamo realizzato un file per ogni pacchetto sotto app.jbomberman.* in tre diversi formati. I file con il nome *simple_NOMEDELPACCHETTO.pdf* contengono il diagramma delle classi in modo semplificato, senza la lista di metodi e campi. I file con il nome *complete_NOMEDELPACCHETTO.pdf* invece contengono i diagrammi delle classi in cui, per ogni classi, vi e' anche la lista di metodi e campi. I file con il nome

complete_NOMEDELPACCHETTO_with_dependencies.pdf contengono anche le indicazioni delle dipendenze tra classi (ad esempio A dipende da B, frecce per indicare incapsulamento e composizione).

Bibliografia

[1] *Design Patterns, Elements of Reusable Object-Oriented Software* di Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides